

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL  
CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE  
SISTEMAS

## GUÍA DE LABORATORIO N.º 03

Aproximación avanzada: Vertex Cover, PTAS y FPTAS

|                            |                                                      |
|----------------------------|------------------------------------------------------|
| <b>Asignatura:</b>         | Algoritmos Avanzados                                 |
| <b>Unidad:</b>             | I. Complejidad y algoritmos de aproximación          |
| <b>Semestre académico:</b> | 2026-II                                              |
| <b>Duración:</b>           | 120 minutos                                          |
| <b>Organización:</b>       | Trabajo grupal en los equipos del proyecto semestral |
| <b>Integrantes:</b>        | Aproximadamente cuatro estudiantes por equipo        |
| <b>Lenguaje sugerido:</b>  | Python 3                                             |
| <b>Docente:</b>            | Héctor Eduardo Ugarte Rojas                          |

**Equipo:** \_\_\_\_\_

**Integrantes:** \_\_\_\_\_

**Fecha:** \_\_\_\_\_

# Índice

|                                                                                      |          |
|--------------------------------------------------------------------------------------|----------|
| <b>1. Objetivo de la práctica</b>                                                    | <b>2</b> |
| <b>2. Trabajo preparatorio</b>                                                       | <b>2</b> |
| <b>3. Materiales</b>                                                                 | <b>3</b> |
| <b>4. Marco teórico breve</b>                                                        | <b>3</b> |
| 4.1. Minimum Vertex Cover . . . . .                                                  | 3        |
| 4.2. Algoritmo 2-aproximado . . . . .                                                | 3        |
| 4.3. Knapsack 0/1 y escalamiento . . . . .                                           | 3        |
| <b>5. Distribución sugerida de los 120 minutos</b>                                   | <b>4</b> |
| <b>6. Ejercicios resueltos</b>                                                       | <b>4</b> |
| 6.1. Ejercicio resuelto 1: validar una cobertura . . . . .                           | 4        |
| 6.2. Ejercicio resuelto 2: algoritmo 2-aproximado . . . . .                          | 4        |
| 6.3. Ejercicio resuelto 3: solución óptima para grafos pequeños . . . . .            | 5        |
| <b>7. Ejercicios propuestos</b>                                                      | <b>6</b> |
| 7.1. Ejercicio propuesto 1: evaluación de Vertex Cover . . . . .                     | 6        |
| 7.2. Ejercicio propuesto 2: aproximación de Knapsack mediante escalamiento . . . . . | 6        |
| <b>8. Producto de entrega</b>                                                        | <b>7</b> |
| <b>9. Tabla de evaluación</b>                                                        | <b>8</b> |
| <b>10. Referencias</b>                                                               | <b>8</b> |

## 1. Objetivo de la práctica

### Objetivo

Implementar y evaluar el algoritmo 2-aproximado para Minimum Vertex Cover, validando formalmente sus salidas y comparándolas con el óptimo en instancias pequeñas. Además, analizar experimentalmente el efecto del parámetro de precisión  $\varepsilon$  en un esquema de aproximación para Knapsack 0/1 basado en escalamiento de valores y Programación Dinámica.

Al finalizar la práctica, el equipo será capaz de:

1. representar grafos no dirigidos y verificar coberturas de vértices;
2. construir un matching maximal y obtener una cobertura 2-aproximada;
3. calcular una cobertura óptima para grafos pequeños mediante búsqueda exacta;
4. medir la razón observada  $|C|/\text{OPT}$ ;
5. implementar una Programación Dinámica exacta para Knapsack basada en valores;
6. aplicar escalamiento para obtener soluciones aproximadas;
7. distinguir evidencia experimental y garantía teórica;
8. relacionar  $\varepsilon$  con calidad, tiempo y tamaño de la tabla dinámica.

## 2. Trabajo preparatorio

Antes del laboratorio, cada integrante deberá:

1. revisar los conceptos de grafo, arista, conjunto, matching y cobertura de vértices;
2. revisar la garantía del algoritmo de Vertex Cover:

$$|C| = 2|M| \leq 2\text{OPT};$$

3. revisar las definiciones de PTAS, FPTAS y algoritmo pseudo-polinomial;
4. recordar la formulación de Knapsack 0/1;
5. responder brevemente:
  - a) ¿Cuál es la diferencia entre matching maximal y matching máximo?
  - b) ¿Por qué toda cobertura debe incluir al menos un extremo de cada arista de un matching?
  - c) ¿Por qué un tiempo  $n^{1/\varepsilon}$  puede corresponder a un PTAS, pero no necesariamente a un FPTAS?
  - d) ¿Qué se espera que ocurra si  $\varepsilon$  disminuye?

### 3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE y terminal.
- Biblioteca estándar: `itertools`, `random`, `time`, `statistics`, `csv` y `math`.
- Git y repositorio del equipo.
- Herramienta para producir tablas y gráficos.
- Instancias de grafos e instancias de Knapsack generadas reproduciblemente.

### 4. Marco teórico breve

#### 4.1. Minimum Vertex Cover

Dado un grafo no dirigido  $G = (V, E)$ , una cobertura de vértices es un conjunto  $C \subseteq V$  tal que:

$$\forall (u, v) \in E, \quad u \in C \vee v \in C.$$

Minimum Vertex Cover busca minimizar  $|C|$ .

#### 4.2. Algoritmo 2-aproximado

El algoritmo selecciona una arista  $(u, v)$ , agrega ambos extremos a  $C$  y elimina todas las aristas incidentes en  $u$  o  $v$ . Las aristas seleccionadas forman un matching  $M$ . Toda cobertura necesita al menos un vértice distinto por cada arista de  $M$ , por lo que  $|M| \leq \text{OPT}$ . Como se agregan dos extremos por arista:

$$|C| = 2|M| \leq 2 \text{OPT}.$$

#### 4.3. Knapsack 0/1 y escalamiento

Cada objeto  $i$  tiene peso  $w_i$  y valor  $v_i$ . Se desea maximizar:

$$\sum_{i=1}^n v_i x_i$$

sujeto a:

$$\sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

Para elaborar un FPTAS se escalan los valores. Después de descartar objetos con  $w_i > W$ , se define:

$$V_{\max} = \max_i v_i, \quad K = \frac{\varepsilon V_{\max}}{n}, \quad v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

La Programación Dinámica trabaja con los valores escalados. La pérdida total queda acotada por  $nK = \varepsilon V_{\max} \leq \varepsilon \text{OPT}$ .

## 5. Distribución sugerida de los 120 minutos

| Etapas      | Actividad                                        | Tiempo |
|-------------|--------------------------------------------------|--------|
| Inicio      | Organización y revisión del trabajo preparatorio | 10 min |
| Resuelto 1  | Validación de una cobertura de vértices          | 15 min |
| Resuelto 2  | Algoritmo 2-aproximado y matching maximal        | 22 min |
| Resuelto 3  | Óptimo exacto y razón observada                  | 20 min |
| Propuesto 1 | Evaluación experimental de Vertex Cover          | 23 min |
| Propuesto 2 | Escalamiento y aproximación de Knapsack          | 20 min |
| Cierre      | Revisión individual y organización de evidencias | 10 min |

## 6. Ejercicios resueltos

### 6.1. Ejercicio resuelto 1: validar una cobertura

Se representará cada arista como una tupla  $(u, v)$  y la cobertura como un conjunto.

Listing 1: Validación de una cobertura de vértices

```

1 def es_vertex_cover(vertices, aristas, cobertura):
2     cobertura = set(cobertura)
3
4     if not cobertura.issubset(set(vertices)):
5         return False
6
7     for u, v in aristas:
8         if u not in cobertura and v not in cobertura:
9             return False
10    return True
11
12
13 vertices = {"a", "b", "c", "d", "e"}
14 aristas = [("a", "b"), ("b", "c"),
15            ("c", "d"), ("d", "e")]
16
17 print(es_vertex_cover(vertices, aristas, {"b", "d"}))
18 print(es_vertex_cover(vertices, aristas, {"a", "c"}))

```

La cobertura  $\{b, d\}$  es válida. El conjunto  $\{a, c\}$  no cubre la arista  $(d, e)$ .

#### Validación

La cobertura se verifica sobre las aristas originales. Las aristas eliminadas durante el algoritmo aproximado no deben perderse para la comprobación final.

### 6.2. Ejercicio resuelto 2: algoritmo 2-aproximado

Una implementación sencilla recorre la lista de aristas. Si ninguno de sus extremos fue seleccionado previamente, elige la arista y agrega ambos extremos.

Listing 2: Aproximación de factor 2 para Vertex Cover

```

1 def vertex_cover_2_aprox(vertices, aristas):
2     cobertura = set()

```

```

3     matching = []
4
5     for u, v in aristas:
6         if u not in cobertura and v not in cobertura:
7             matching.append((u, v))
8             cobertura.add(u)
9             cobertura.add(v)
10
11     assert es_vertex_cover(vertices, aristas, cobertura)
12     return cobertura, matching
13
14
15 vertices = {"a", "b", "c", "d", "e"}
16 aristas = [("a", "b"), ("b", "c"),
17            ("c", "d"), ("d", "e")]
18
19 cobertura, matching = vertex_cover_2_aprox(vertices, aristas)
20 print("Matching:", matching)
21 print("Cobertura:", cobertura)
22 print("Tamaño:", len(cobertura))

```

Con el orden mostrado se seleccionan  $(a, b)$  y  $(c, d)$ . Por tanto:

$$M = \{(a, b), (c, d)\}, \quad C = \{a, b, c, d\}.$$

Se obtiene  $|M| = 2$  y  $|C| = 4$ .

#### Por qué funciona la condición

Una arista se selecciona solo cuando ninguno de sus extremos está en la cobertura. Como cada arista seleccionada agrega ambos extremos, las aristas seleccionadas posteriormente no pueden compartir extremos con las anteriores.

### 6.3. Ejercicio resuelto 3: solución óptima para grafos pequeños

Para evaluar la calidad observada se calculará el óptimo por enumeración de subconjuntos. Esta solución solo es apropiada para grafos pequeños.

Listing 3: Minimum Vertex Cover exacto por enumeración

```

1 from itertools import combinations
2
3
4 def vertex_cover_exacto(vertices, aristas):
5     lista = sorted(vertices)
6
7     for k in range(len(lista) + 1):
8         for candidatos in combinations(lista, k):
9             if es_vertex_cover(vertices, aristas, candidatos):
10                 return set(candidatos)
11
12     return set()
13
14
15 optimo = vertex_cover_exacto(vertices, aristas)
16 aprox, matching = vertex_cover_2_aprox(vertices, aristas)
17 razon = len(aprox) / len(optimo) if optimo else 1.0
18
19 print("Optimo:", optimo, len(optimo))

```

```
20 print("Aproximado:", aprox, len(aprox))
21 print("Razon observada:", razon)
22 assert len(aprox) <= 2 * len(optimo)
```

Para el camino de cinco vértices:

$$C^* = \{b, d\}, \quad |C^*| = 2,$$

y la ejecución aproximada anterior produce cuatro vértices. Entonces:

$$r(I) = \frac{|C|}{|C^*|} = \frac{4}{2} = 2.$$

### Complejidad

La búsqueda exacta puede examinar hasta  $2^{|V|}$  subconjuntos. Su propósito es obtener una referencia para instancias pequeñas, no sustituir al algoritmo aproximado en grafos grandes.

## 7. Ejercicios propuestos

### 7.1. Ejercicio propuesto 1: evaluación de Vertex Cover

Genere o seleccione por lo menos 30 grafos pequeños, variando el número de vértices y la densidad. Para cada grafo ejecute el algoritmo aproximado y la solución exacta.

#### Requisitos:

1. utilizar semillas reproducibles para los grafos aleatorios;
2. evitar lazos y normalizar las aristas no dirigidas;
3. comprobar la factibilidad de cada cobertura;
4. registrar  $|V|$ ,  $|E|$ ,  $|M|$ ,  $|C|$ , OPT y  $r(I)$ ;
5. comprobar experimentalmente  $|C| \leq 2 \text{ OPT}$ ;
6. ejecutar al menos 20 órdenes aleatorios de aristas para un mismo grafo;
7. reportar cuántas veces se obtiene el óptimo;
8. identificar la peor razón observada;
9. comparar tiempos del método exacto y del aproximado;
10. incluir grafos camino, ciclo, estrella, completo y aleatorio.

**Pregunta de análisis.** ¿El orden de las aristas cambia la cobertura obtenida, su tamaño o ambas propiedades? Explique por qué la garantía 2 permanece válida.

### 7.2. Ejercicio propuesto 2: aproximación de Knapsack mediante escalamiento

Implemente una Programación Dinámica exacta basada en valores y una versión aproximada que escale los valores mediante  $\epsilon$ .

#### Esquema de trabajo:

1. descartar objetos con peso mayor que la capacidad;
2. calcular  $V_{\text{máx}}$  y  $K = \varepsilon V_{\text{máx}}/n$ ;
3. calcular  $v'_i = \lfloor v_i/K \rfloor$ ;
4. ejecutar la DP sobre los valores escalados;
5. reconstruir los objetos seleccionados;
6. evaluar la solución con los valores originales.

**Requisitos:**

1. utilizar por lo menos 20 instancias pequeñas con óptimo conocido;
2. evaluar  $\varepsilon \in \{0.50, 0.25, 0.10, 0.05\}$ ;
3. comprobar peso total, valor original y factibilidad;
4. registrar  $\text{ALG}_\varepsilon$ ,  $\text{OPT}$ ,  $\text{ALG}_\varepsilon / \text{OPT}$ , tiempo y estados procesados;
5. comprobar la garantía observada  $\text{ALG}_\varepsilon \geq (1 - \varepsilon) \text{OPT}$ ;
6. generar un gráfico de calidad frente a  $\varepsilon$ ;
7. generar un gráfico de tiempo o estados frente a  $1/\varepsilon$ ;
8. explicar por qué reducir  $\varepsilon$  normalmente mejora la calidad y aumenta el costo.

**Cuidado numérico**

Si  $K = 0$  debido a una instancia vacía o sin objetos factibles, el caso debe tratarse antes del escalamiento. Las operaciones de redondeo deben ser reproducibles y la solución debe evaluarse siempre con los valores originales.

## 8. Producto de entrega

**Producto de entrega**

El equipo entregará:

1. código fuente organizado y ejecutable;
2. implementación 2-aproximada y exacta de Vertex Cover;
3. implementación exacta y aproximada de Knapsack;
4. validadores de factibilidad para ambos problemas;
5. instancias o generadores reproducibles;
6. gráficos de calidad y rendimiento;
7. respuestas a las preguntas de análisis;
8. respuesta a las preguntas de trabajo preparatorio y preguntas de análisis.
9. conclusiones grupales de 200 a 300 palabras.
10. informe en PDF con lo anterior, incluyendo ejercicios resueltos y propuestos.

## 9. Tabla de evaluación

| Criterio                                     | Indicador de logro                                                                                                                                                                             | Puntaje   |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Asistencia y participación en el laboratorio | Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.                                                             | 4         |
| Corrección y funcionamiento                  | Las implementaciones aproximadas y exactas funcionan, producen soluciones factibles, calculan correctamente las métricas y consideran todos los casos solicitados.                             | 8         |
| Análisis de complejidad y resultados         | Justifica complejidades, interpreta las razones observadas y el efecto de $\varepsilon$ , diferencia evidencia experimental y garantía teórica, y reconoce los límites de los métodos exactos. | 3         |
| Dominio individual                           | Explica el algoritmo 2-aproximado, la demostración del factor 2, PTAS/FPTAS y escalamiento; predice resultados y realiza una prueba o modificación breve.                                      | 4         |
| <b>Total</b>                                 |                                                                                                                                                                                                | <b>19</b> |

## 10. Referencias

### Referencias

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.<sup>a</sup> ed.). MIT Press.
- [2] Vazirani, V. V. (2001). *Approximation Algorithms*. Springer.
- [3] Williamson, D. P. y Shmoys, D. B. (2011). *The Design of Approximation Algorithms*. Cambridge University Press.
- [4] Massachusetts Institute of Technology. *Design and Analysis of Algorithms: Approximation Algorithms*. MIT OpenCourseWare.
- [5] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>